Cloud Agents API and Webhooks

Module 8 · Day 2 (Hands-On)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise
Prerequisites	User API key (Module 7), Python 3.8+, ngrok installed, GitHub repository
Module Goal	Programmatically create, stream, and manage Cloud Agents, and set up webhook notifications

Learning Objectives

By the end of this module, participants will be able to:

- Create a Cloud Agent programmatically using the API
- Stream agent responses in real-time using SSE with resume support
- List and download artifacts from a completed agent
- Create a webhook endpoint with HMAC verification
- Test webhooks locally using ngrok
- Build an end-to-end automated agent workflow

Agenda

Lesson	Topic	Time
8.1	Creating a Cloud Agent Programmatically	15 min
8.2	Streaming Agent Responses (SSE)	15 min
8.3	Listing and Downloading Artifacts	15 min
8.4	Creating a Webhook Endpoint	15 min
8.5	Testing Webhooks Locally with ngrok	13 min
8.6	End-to-End Automated Agent Workflow	17 min

Lesson 8.1

Creating a Cloud Agent Programmatically

Concept · 5 min · Exercise · 10 min

Agent + Runs

Concept	Description
Agent	Durable entity with conversation history and workspace state
Run	Single execution (one prompt/response cycle)

Key endpoint: `POST /v1/agents`

Request Fields

Required:

Field	Example
`prompt.text`	"Add a README.md file"
`repos[].url`	"https://github.com/org/repo"

Optional: `autoCreatePR` · `model.id` · `webhookUrl` · `webhookSecret`

Exercise 8.1 — Create with curl

```
export CURSOR_USER_API_KEY="cursor_xxxxxxxxxxx"

curl -X POST https://api.cursor.com/v1/agents \
  -u "$CURSOR_USER_API_KEY:" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": {"text": "Add a README.md file with setup instructions"},
    "repos": [{"url": "https://github.com/YOUR_ORG/YOUR_REPO", "startingRef": "main"}],
    "autoCreatePR": true
  }' | jq '.'
```

Exercise 8.1 — Capture IDs

```
AGENT_ID=$(echo "$RESPONSE" | jq -r '.agent.id')
RUN_ID=$(echo "$RESPONSE" | jq -r '.run.id')

echo "Agent ID: $AGENT_ID"
echo "Dashboard: https://cursor.com/agents/$AGENT_ID"
```

Create with specific model: `"model": {"id": "claude-4.7-opus"}`

Exercise 8.1 — Python Helper

```python
def create_agent(prompt, repo_url, auto_create_pr=False, model=None):
    payload = {
        "prompt": {"text": prompt},
        "repos": [{"url": repo_url}],
        "autoCreatePR": auto_create_pr
    }
    if model:
        payload["model"] = {"id": model}
    response = requests.post(f"{BASE_URL}/agents", auth=AUTH, json=payload)
    data = response.json()
    return data["agent"]["id"], data["run"]["id"]
```

Success Criteria: Agent created · IDs captured · appears in dashboard · Python function works

Lesson 8.2

Streaming Agent Responses (SSE)

Concept · 5 min · Exercise · 10 min

SSE Event Types

Event	When It Happens	Data Example
`status`	Run status changes	`{"status":"RUNNING"}`
`assistant`	Agent speaks	`{"text":"I'll read the file..."}`
`thinking`	Agent is reasoning	`{"text":"Let me consider..."}`
`tool_call`	Agent uses a tool	`{"name":"read_file","status":"started"}`
`result`	Run completes	`{"status":"FINISHED"}`
`error`	Something went wrong	`{"message":"..."}`
`done`	Stream ends	`{}`

Resume Support

SSE streams support the `Last-Event-ID` header — if your connection drops, resume from the last received event.

Exercise 8.2 — Stream with curl

```
curl -N -u "$CURSOR_USER_API_KEY:" \
  -H "Accept: text/event-stream" \
  "https://api.cursor.com/v1/agents/$AGENT_ID/runs/$RUN_ID/stream"
```

Parse lines starting with `event:` and `data:` — print assistant text, tool calls, and result status.

Exercise 8.2 — Python SSE Client

```python
def stream_agent_response(agent_id, run_id, on_event=None):
    url = f"{BASE_URL}/agents/{agent_id}/runs/{run_id}/stream"
    response = requests.get(url, auth=AUTH, stream=True)
    for line in response.iter_lines():
        if line.startswith(b'event:'):
            current_event = line[6:].strip().decode()
        elif line.startswith(b'data:'):
            data = json.loads(line[5:].strip())
            if on_event:
                on_event(current_event, data)
```

Exercise 8.2 — ResumableSSEClient

Track `last_event_id` from `id:` lines → send as `Last-Event-ID` header on reconnect

Also: `stream_to_file()` saves full SSE log for later review

Success Criteria: Stream connected · received events · Python client works · resume implemented

Lesson 8.3

Listing and Downloading Artifacts

Concept · 5 min · Exercise · 10 min

Key Endpoints

Endpoint	Method	Purpose
`/v1/agents/{id}/artifacts`	GET	List all artifacts
`/v1/agents/{id}/artifacts/download`	GET	Get presigned URL for download

Important: Download URLs expire after **15 minutes**.

Exercise 8.3 — Wait & List

```python
def wait_for_completion(agent_id, timeout=300, poll_interval=5):
    while time.time() - start < timeout:
        status = get_agent_status(agent_id).get('status')
        if status == 'FINISHED': return True
        elif status == 'ERROR': return False
        time.sleep(poll_interval)

def list_artifacts(agent_id):
    response = requests.get(f"{BASE_URL}/agents/{agent_id}/artifacts", auth=AUTH)
    return response.json().get('items', [])
```

Exercise 8.3 — Download

Single artifact:

```
response = requests.get(
    f"{BASE_URL}/agents/{agent_id}/artifacts/download",
    auth=AUTH, params={"path": artifact_path}
)
download_url = response.json().get('url')
# curl download_url → save to disk
```

All artifacts: loop items, create subdirs, download each via presigned URL

Exercise 8.3 — CI Integration

```python
def process_test_results(agent_id):
    wait_for_completion(agent_id, timeout=600)
    download_artifact(agent_id, "artifacts/junit.xml", "test_results.xml")
    # Parse XML → exit 1 if failures/errors, else exit 0
```

Success Criteria: Listed artifacts · downloaded single + all · CI workflow integration

Lesson 8.4

Creating a Webhook Endpoint

Concept · 5 min · Exercise · 10 min

Webhook Headers

Header	Description
`X-Webhook-Signature`	HMAC-SHA256 signature for verification
`X-Webhook-ID`	Unique delivery ID
`X-Webhook-Event`	Event type (`statusChange`)

Webhook Payload

```
{
  "event": "statusChange",
  "id": "agent_abc123",
  "status": "FINISHED",
  "source": {"repository": "https://github.com/your-org/your-repo"},
  "target": {"url": "...", "prUrl": "https://github.com/.../pull/123"},
  "summary": "Added README.md and fixed tests"
}
```

Exercise 8.4 — HMAC Verification

```python
def verify_signature(raw_body, signature_header):
    received = signature_header[7:]  # strip "sha256="
    expected = hmac.new(
        WEBHOOK_SECRET.encode(), raw_body, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, received)
```

Flask route: verify signature → parse payload → handle FINISHED/ERROR

Exercise 8.4 — Configure Agent

```
curl -X POST https://api.cursor.com/v1/agents \
  -u "$CURSOR_USER_API_KEY:" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": {"text": "Add a CONTRIBUTING.md file"},
    "repos": [{"url": "https://github.com/YOUR_ORG/YOUR_REPO"}],
    "webhookUrl": "https://your-domain.com/webhook/cursor",
    "webhookSecret": "your-secret-here",
    "autoCreatePR": true
  }'
```

Success Criteria: Server running · signature verified · payload parsed · agent configured

Lesson 8.5

Testing Webhooks Locally with ngrok

Concept · 5 min · Exercise · 8 min

What Is ngrok?

Creates a secure tunnel from a public URL to your local server.

- Test webhooks without deploying
- Debug locally · Demo to stakeholders

Exercise 8.5 — Steps 1–3

Step 1: Start tunnel:

```
ngrok http 5000
# Forwarding: https://abc123.ngrok.io -> http://localhost:5000
```

Step 2: Copy HTTPS URL

Step 3: Create agent with ngrok URL:

```
curl -X POST https://api.cursor.com/v1/agents ... \
  -d '{"webhookUrl": "https://abc123.ngrok.io/webhook/cursor", ...}'
```

Exercise 8.5 — Inspect & Replay

Step 4: Inspect requests at `http://127.0.0.1:4040`

Step 5: Replay failed webhooks (ngrok premium) — inspect raw body and headers

Success Criteria: Tunnel established · webhook received · signature verified · inspected in ngrok UI

Lesson 8.6

End-to-End Automated Agent Workflow

Concept · 5 min · Exercise · 12 min

The Capstone Integration

Combine everything into `automated_workflow.py`:

1. **Create agent** (with optional webhook URL)
2. **Wait for completion** (webhook or polling)
3. **Download artifacts**
4. **Process results** (CI exit codes, notifications)

Workflow Architecture

```
create_agent() → wait (webhook OR polling) → download_artifacts()
       ↑                    ↑
  Flask webhook server   completion_event.set()
  (background thread)    on FINISHED status
```

Run the Workflow

```
export CURSOR_USER_API_KEY="cursor_xxxxxxxxxxx"

python automated_workflow.py \
  --repo "https://github.com/YOUR_ORG/YOUR_REPO" \
  --prompt "Add a comprehensive README.md with setup and API docs" \
  --output "./outputs"

# Polling only (no webhook):
python automated_workflow.py --repo "..." --prompt "..." --no-webhook
```

Workflow Output

```
🚀 CLOUD AGENT AUTOMATED WORKFLOW
📝 Creating agent... Agent ID: agt_abc123
⏳ Waiting for completion...
✅ Webhook received: Agent agt_abc123 completed
📎 Downloaded 3 artifacts to agent_outputs/
✅ WORKFLOW COMPLETE
```

Success Criteria: Creates agent · waits (webhook/polling) · downloads artifacts · end-to-end run

Module Summary

Lesson	Topic	Key Skill
8.1	Creating a Cloud Agent	Programmatic agent launch
8.2	Streaming Agent Responses	SSE with resume support
8.3	Listing and Downloading Artifacts	CI pipeline integration
8.4	Creating a Webhook Endpoint	HMAC verification
8.5	Testing Webhooks with ngrok	Local tunnel debugging
8.6	End-to-End Workflow	Complete automation

Quick Reference Card

```
ENDPOINTS:
  POST /v1/agents                         Create agent
  GET  /v1/agents/{id}                    Get status
  GET  /v1/agents/{id}/runs/{id}/stream   SSE stream
  GET  /v1/agents/{id}/artifacts          List artifacts
  GET  /v1/agents/{id}/artifacts/download Download

WEBHOOK:  X-Webhook-Signature (HMAC-SHA256)  |  X-Webhook-Event

SSE:  status · assistant · thinking · tool_call · result · error · done

NGROK:  ngrok http 5000  |  inspect at http://127.0.0.1:4040
```

Up Next: Module 9

Admin and Analytics APIs

> Now that you can programmatically launch and monitor agents, **Module 9** covers team management and usage insights.

End of Module 8